Marwadi University Marwadi Chandarana Group NAAC A+	**Marwadi University** **Faculty of Engineering and Technology** **Department of Information and Communication Technology**	
Design and Analysis of Algorithm (01CT0512)	**Aim: Implementing the Sorting Algorithms and understanding the time and space complexities**	
Experiment No: 1	**Date: 9/4/2026**	**Enrolment No: 92301733059**

Insertion Sort

Theory:

- Insertion Sort is a simple comparison-based sorting algorithm that builds the final sorted array one element at a time.
- It starts with the second element and compares it with the previous elements, inserting it into its correct position.
- For every element, it shifts larger elements to the right until the correct spot for the current element is found.
- It is similar to how people arrange playing cards in hand.

Programming Language: C++

Code:

```cpp
//Insertion Sort
#include <iostream>
#include <vector>
using namespace std;

void printVector(const vector<int>& arr) {
    for (int value : arr) {
        cout << value << " ";
    }
    cout << endl;
}

void insertionSort(vector<int>& arr) {
    int n = arr.size();
    for (int i = 1; i < n; ++i) {
        int key = arr[i];
        int j = i - 1;

```

```
        while (j >= 0 && arr[j] > key) {
            arr[j + 1] = arr[j];
            j = j - 1;
        }
        arr[j + 1] = key;
    }
}

int main(){
    vector<int> data = {12, 11, 13, 5, 6};

    cout << "Unsorted array: ";
    printVector(data);

    insertionSort(data);

    cout << "Sorted array:   ";
    printVector(data);

    return 0;
}
```

Output

```
[Running] cd "c:\Users\Mo7am\Downloads\Data-Structure-Algor
"c:\Users\Mo7am\Downloads\Data-Structure-Algorithms-main\Da
Unsorted array: 12 11 13 5 6
Sorted array:   5 6 11 12 13

[Done] exited with code=0 in 10.418 seconds
```

Space complexity: O(1)

Insertion Sort is an in-place sorting algorithm, meaning it does not require any extra memory apart from a few variables (key, j). It modifies the original array directly, so the extra space used is constant.

Time complexity:

Best case time complexity O(n) : In the best case, the array is already sorted. The inner while loop never executes because no shifting is needed. Only one comparison per element is performed → total linear time.

Worst case time complexity O(n²) : In the worst case, the array is reverse sorted.

Each element must be compared with all previous elements and shifted completely. Total comparisons and shifts = $\mathbf{1 + 2 + 3 + \cdots + (n-1) = n(n-1)/2}$.

Bubble Sort

Theory:

- Bubble Sort is the simplest sorting algorithm where adjacent elements are repeatedly compared and swapped if they are in the wrong order.
- After the first pass, the largest element “bubbles up” to the last position.
- The process repeats for the remaining elements until the array is sorted.
- An optimized version uses a flag to stop if no swaps are made in a pass, which improves best-case performance.

Programming Language: C++

Code:

```cpp
//Bubble Sort
#include <iostream>
using namespace std;

void swap(int* a, int* b){
    int temp = *a;
    *a = *b;
    *b = temp;
}

void bubbleSort(int array[], int size){
    int i,j;
    for(i=0; i<size-1; i++){
        for(j=0; j<size-1-i;j++){
            if(array[j+1] < array[j]){
                swap(array[j+1], array[j]);
            }
        }
    }
}

```

```cpp

int main(){
    int array[] = {1,5,4,3,8,20,0,2};
    int size = sizeof(array)/sizeof(array[0]);
    bubbleSort(array, size);
    for(int i=0; i<size;i++) cout << array[i] <<" ";
    cout <<"\n";
    return 0;
}
```

Output

```
[Running] cd "c:\Users\Mo7am\Downloads\Data-Structure-Algorithms-
"c:\Users\Mo7am\Downloads\Data-Structure-Algorithms-main\Data-Str
0 1 2 3 4 5 8 20

[Done] exited with code=0 in 1.617 seconds
```

Space complexity O(1) : Bubble Sort is an in-place sorting algorithm. It only uses a few extra variables (swapped, loop counters) and does not require any additional memory proportional to input size.

Time complexity:

Best case time complexity O(n) : In the best case, the array is already sorted. Due to the optimized flag (swapped), no swaps occur in the first pass, so the algorithm stops early after one iteration → linear time.

Worst case time complexity O(n^2) : In the worst case, the array is reverse sorted. Each element must be compared and swapped repeatedly across multiple passes.

Selection Sort

Theory:

- Selection Sort divides the array into two parts: sorted and unsorted.
- In each iteration, it selects the smallest element from the unsorted part and swaps it with the first element of the unsorted part. This process repeats, expanding the sorted portion until the array is fully sorted.
- The number of comparisons is always fixed, regardless of the initial order of elements.

Programming Language: C++

Code:

```cpp
//Selection Sort
#include <iostream>
#include <vector>
#include <utility>
using namespace std;

void printVector(const vector<int>& arr) {
    for (int value : arr) {
        cout << value << " ";
    }
    cout << endl;
}

void selectionSort(vector<int>& arr) {
    int n = arr.size();

    for (int i = 0; i < n - 1; ++i) {
        int min_index = i;
        for (int j = i + 1; j < n; ++j) {
            if (arr[j] < arr[min_index]) {
                min_index = j;
            }
        }

        if (min_index != i) {
            swap(arr[min_index], arr[i]);
        }
    }
}

int main() {
    vector<int> data = {64, 25, 12, 22, 11};

```

```
        cout << "Unsorted array: ";
        printVector(data);

        selectionSort(data);

        cout << "Sorted array:   ";
        printVector(data);

        return 0;
    }
```

Output

```
[Running] cd "c:\Users\Mo7am\Downloads\Data-Structure-Algorithms-main\D
"c:\Users\Mo7am\Downloads\Data-Structure-Algorithms-main\Data-Structure
Unsorted array: 64 25 12 22 11
Sorted array:   11 12 22 25 64

[Done] exited with code=0 in 1.584 seconds
```

Space complexity O(1) : Selection Sort is an in-place sorting algorithm. It only uses a few extra variables (min_idx, loop counters) and does not require any additional memory proportional to the input size.

Time complexity:

Best case time complexity O(n²) : Even if the array is already sorted, Selection Sort still scans the entire unsorted portion to find the minimum element in every iteration. Number of comparisons remains fixed $(n - 1) + (n - 2) + \cdots + 1 = n(n - 1) / 2$

Worst case time complexity O(n²) : In the worst case (reverse sorted or random order), the algorithm still performs the same number of comparisons as in the best case. Although swaps may vary, comparisons dominate → overall $O(n^2)$.

Counting Sort

Theory:

- Counting Sort is a non-comparison-based algorithm used for sorting integers or objects with keys that can be mapped to integers.

- It works by counting the occurrences of each unique value in the array. Then, using cumulative counts, it determines the correct positions of elements in the output array.
- Finally, elements are placed according to their counts to get a sorted result.

Programming Language: C++

Code:

```cpp
#include <iostream>
#include <vector>
#include <algorithm> // For std::max_element
using namespace std;

void printVector(const vector<int>& arr) {
    for (int val : arr) {
        cout << val << " ";
    }
    cout << std::endl;
}

void countingSort(std::vector<int>& arr) {
    if (arr.empty()) {
        return;
    }

    // 1. Find the maximum element to determine the range
    int max = *std::max_element(arr.begin(), arr.end());

    // 2. Create a count array and store the frequency of each element
    vector<int> count(max + 1, 0);
    for (int val : arr) {
        count[val]++;
    }

    // 3. Calculate cumulative counts
    for (int i = 1; i <= max; ++i) {
        count[i] += count[i - 1];
    }

    // 4. Build the output array
    vector<int> output(arr.size());
```

```cpp
        vector<int> output(arr.size());
        // Iterate from the end to maintain stability
        for (int i = arr.size() - 1; i >= 0; --i) {
            int val = arr[i];
            output[count[val] - 1] = val;
            count[val]--;
        }

        // 5. Copy the sorted elements back to the original array
        arr = output;
    }

int main() {
        vector<int> data = {4, 2, 2, 8, 3, 3, 1};

        cout << "Unsorted array: ";
        printVector(data);

        countingSort(data);

        cout << "Sorted array:   ";
        printVector(data);

        return 0;
    }
```

Output

```
[Running] cd "c:\Users\Mo7am\Downloads\Data-Structure-Algorithms-main\Data-Structure-Algorithms-main\"
"c:\Users\Mo7am\Downloads\Data-Structure-Algorithms-main\Data-Structure-Algorithms-main\"countingSort
Unsorted array: 4 2 2 8 3 3 1
Sorted array:   1 2 2 3 3 4 8

[Done] exited with code=0 in 2.355 seconds
```

Space complexity O(n + k) : Counting Sort requires:

- An input array of size n
- A count array of size k + 1 (where k = maximum element)

Thus, total extra space depends on both input size and range of values O(n + k).

Time complexity:

Best case time complexity O(n + k) : Counting Sort does not depend on input order (sorted or unsorted). Steps involved:

- Finding max element →O(n)
- Counting occurrences → O(n)
- Traversing count array → O(k) \ Total = O(n + k).

Worst case time complexity O(n + k) : Even in the worst case, Counting Sort performs the same steps:

- Count frequencies → O(n)
- Iterate through count array → O(k)

No comparisons are involved, so complexity remains O(n + k) regardless of input arrangement.

Observation

Sorting algorithms are the bread and butter of computational logic, each with its own "personality" and efficiency profile. Based on standard experimental observations of these four algorithms.

Comparison Table

Feature	Bubble Sort	Selection Sort	Insertion Sort	Counting Sort
Best Case Time	O(n) (Optimized)	O(n^2)	O(n)	O(n+k)
Average Case Time	O(n^2)	O(n^2)	O(n^2)	O(n+k)
Worst Case Time	O(n^2)	O(n^2)	O(n^2)	O(n+k)
Space Complexity	O(1) (In-place)	O(1) (In-place)	O(1) (In-place)	O(k) (Extra space)
Stability	Stable	Unstable	Stable	Stable
Data Type	Any comparable	Any comparable	Any comparable	Integers only

Conclusion

1. **Efficiency vs. Range:** If you are dealing with a small range of integers, Counting Sort is unbeatable in speed. However, for general-purpose sorting of objects or floating-point numbers, comparison-based sorts are mandatory.

2. **The "Small Data" King**: Insertion Sort is the most efficient among the O(n^2) family for small or nearly-sorted arrays due to its low overhead and adaptive nature.

3. **Space Constraints:** If memory is extremely tight and you cannot afford extra arrays, Bubble, Selection, and Insertion are preferred over Counting Sort as they sort "in-place."

4. **Stability Matters:** If you need to maintain the relative order of equal elements (e.g., sorting a list of people by name, then by age), Bubble, Insertion, and Counting Sort are reliable; Selection Sort is generally not.

Marwadi University Marwadi Chandarana Group NAAC A+	**Marwadi University** **Faculty of Engineering and Technology** **Department of Information and Communication Technology**	
Design and Analysis of Algorithm (01CT0512)	**Aim: Implementing the Searching Algorithms & understanding the time and space complexities**	
Experiment No: 2	**Date: 9/4/2026**	**Enrolment No: 92301733059**

Linear Search

Theory:

- Linear Search is the simplest searching algorithm used to find an element in a list or array.
- It works by checking each element one by one from the beginning until the desired element is found or the list ends.
- If the element is found, the position (index) of the element is returned; otherwise, it indicates that the element is not present.
- It does not require the array to be sorted, making it useful for small or unsorted datasets.
- However, it is less efficient compared to other searching algorithms for large datasets since it may need to check every element.

Programming Language: C++

Code:

```cpp
//Linear search
#include <iostream>
using namespace std;
int search(int arr[], int target, int size){
    int i=0;
    while(i<size){
        if(arr[i] == target){
            return i;
        }
        i++;
    }
    return -1;
}
int main(){
    int array[] = {1,4,2,6,4,8,5};
    int size = (sizeof(array)/sizeof(array[0]));
    int x = search(array, 8, size);
    if(x != -1){
        cout << "Target at position:" << x+1 <<endl;
    }
    else cout << "Not found" <<endl;
    return 0;
}   // Done
```

Output

```
[Running] cd "c:\Users\Mo7am\Downloads\Data-Structure-Algorithms-main
"c:\Users\Mo7am\Downloads\Data-Structure-Algorithms-main\Data-Structu
Target at position:6

[Done] exited with code=0 in 1.708 seconds
```

Space complexity O(1): Linear Search uses only a few extra variables (i, target) and does not require any additional memory proportional to input size. Hence, it is a constant space algorithm.

Time complexity

Best case time complexity O(1) : In the best case, the target element is found at the first position of the array. Only one comparison is needed → constant time.

Worst case time complexity O(n) : In the worst case:

- The element is at the last position, or
- The element is not present in the array

So, the algorithm checks all n elements → linear time complexity.

Binary Search

Theory:

- Binary Search is an efficient searching algorithm used to find an element in a sorted array.
- It works by repeatedly dividing the search interval into two halves.
- Initially, it compares the target element with the middle element of the array.
- If the target matches the middle element, the search is successful.
- If the target is smaller than the middle element, the search continues in the left half; otherwise, it continues in the right half.
- This process repeats until the element is found or the search interval becomes empty.
- Binary Search is much faster than Linear Search for large datasets due to its logarithmic time complexity.

Programming Language: C++

Code:

```cpp
#include <iostream>
#include <algorithm>
using namespace std;

int binarySearch(int a[], int left, int right, int target){
    while(left <= right){
        int mid = (left + right)/2;
        if (a[mid] == target){
            return mid;
```

```
                return mid;
            }
            if (target > a[mid]){
                return binarySearch(a, mid+1, right, target);
                // left = mid + 1;
            }
            else{
                return binarySearch(a, left, mid-1, target);
                // right = mid - 1;
            }
        }
        return -1;
    }
    int main(){
        int array[] = {1,4,2,6,4,8,5};
        int size = sizeof(array)/sizeof(array[0]);
        sort(array, array + size);
        int target;
        int low =0, high = size-1;
        cout << "Enter target: ";
        cin >> target;
        int x = binarySearch(array, low, high, target);
        if(x != -1){
            cout << "psition is: " << 1+x;
        } else cout << "Not found";
        cout << endl;
        return 0;
    } //Done
```

Output

```
[Running] cd "c:\Users\Mo7am\Downloads\Data-Structure-Algorithms-main\Data-S
"c:\Users\Mo7am\Downloads\Data-Structure-Algorithms-main\Data-Structure-Algo
Target at position:3

[Done] exited with code=0 in 2.409 seconds

[Running] cd "c:\Users\Mo7am\Downloads\Data-Structure-Algorithms-main\Data-S
"c:\Users\Mo7am\Downloads\Data-Structure-Algorithms-main\Data-Structure-Algo
Target at position:6
```

Space complexity O(1)

The given implementation is iterative, so it only uses a few variables (low, high, mid). No extra memory proportional to input size is used → constant space.

Time complexity

Best case time complexity O(1) : In the best case, the target element is found at the middle position in the first comparison. Only one comparison is needed → constant time.

Worst case time complexity O(log n) : In each step, Binary Search halves the search space. So, the number of comparisons required is: **log 2** n

This happens when the element is at the extreme end or not present → logarithmic time.

Binary Search (Recursive)

Theory

- **Divide and Conquer :** Recursive Binary Search follows the "Divide and Conquer" paradigm. It breaks the problem into smaller sub-problems until a base case is reached.
- **Function Signature:** Unlike the iterative version, the recursive function typically takes the array, the target, and the current low and high indices as parameters.
- **Process:**
  1. Base Case: If low > high, the search interval is empty, and the element is not present.
  2. Calculate the middle element.
  3. If the middle matches the target, return the index.
  4. **Recursive Step:** If the target is smaller, the function calls itself using the range (low, mid - 1). If larger, it calls itself using (mid + 1, high).
- **Efficiency:** It maintains the same logarithmic logic as the iterative method but uses the "call stack" to keep track of the search boundaries.

Code:

```
//Recursive Binary Search
#include <iostream>
#include <algorithm>
using namespace std;
int binarySearch(int a[], int left, int right, int target){
    while(left <= right){
        int mid = (left + right)/2;
        if (a[mid] == target){
            return mid;
        }
        if (target > a[mid]){
            left = mid + 1;
        }
```

```
            else{
                right = mid - 1;
            }
        }
        return -1;
    }
    int main(){
        int array[] = {1,4,2,6,4,8,5};
        int size = sizeof(array)/sizeof(array[0]);
        sort(array, array + size);
        cout << "Sorted Array:";
        for(int i=0; i <size;i++){
            cout << array[i] << " ";
        } cout << endl;
        int target;
        int low =0, high = size-1;
        cout << "Enter target: ";
        cin >> target;

        int x = binarySearch(array, low, high, target);
        if(x != -1){
            cout << "psition is: " << 1+x;
        } else cout << "Not found";
        cout << endl;
        return 0;
    }    //Done
```

Time Complexity

Best Case O(1) : The target is found at the middle index during the very first function call.

Worst Case: O(log n) : The function continues to call itself, halving the search space each time until only one element remains or the range is exhausted. The number of recursive calls is proportional to (log_2 n).

Space Complexity

Worst Case O(log n) : Unlike the iterative method O(1), the recursive method requires space on the call stack. Each time the function calls itself, a new "stack frame" is created to store the variables and return address. Since there are (log n) recursive calls in the worst case, the space complexity becomes logarithmic.

Conclusion

1. **Efficiency :** Binary Search is significantly faster than Linear Search for large datasets due to its logarithmic time complexity (O(log n)). For example, in a list of 1,024 elements, Linear Search might take 1,024 comparisons, while Binary Search takes a maximum of 10.

2. **Versatility:** Linear Search is more versatile in terms of data organization since it does not require sorting. It is the preferred choice for small lists or unsorted data where the overhead of sorting would exceed the search time.
3. **Memory Trade-offs**: Both iterative versions are highly memory-efficient (O(1)). However, when implementing Recursive Binary Search, one must account for the O(log n) space used by the system's call stack.
4. **Final Verdict:** Use Linear Search for small, unsorted, or frequently changing data. Use Binary Search for large, static, or pre-sorted datasets to achieve optimal performance.

Marwadi University Marwadi Chandarana Group NAAC A+	**Marwadi University** **Faculty of Engineering and Technology** **Department of Information and Communication Technology**	
Design and Analysis of Algorithm (01CT0512)	**Aim: Implementing the Exponential (Power) function to calculate xn using Divide and Conquer Approach**	
Experiment No: 3	**Date: 9/4/2026**	**Enrolment No: 92301733059**

Theory:

The Exponential (Power) function calculates xn where xxx is the base and n is a non- negative integer exponent. There are multiple approaches to compute this, each with different time complexities.

1. Exponential Function using Iterative (Naive) Approach

This approach multiplies the base x with itself n times sequentially.

Algorithm:

- Initialize result = 1.
- Repeat n times: multiply result by xxx.
- Return result.

Time Complexity:

Best, Average, Worst Case: O(n) (requires n multiplications).

Space Complexity:

O(1) (uses only a few variables).

Use Case:

- Simple to implement.
- Suitable when n is small.

```
#include <iostream>
using namespace std;
/**
 * Calculates a^b using the Naive Iterative Approach.
 * Time Complexity: O(b)
 * Space Complexity: O(1)
 */
double power(double base, int exponent) {
    double result = 1.0;
```

```
    // Handle negative exponents
    int absExponent = (exponent < 0) ? -exponent : exponent;

    for (int i = 0; i < absExponent; i++) {
        result *= base;
    }

    // If exponent was negative, return the reciprocal
    return (exponent < 0) ? 1.0 / result : result;
}
int main() {
    double b;
    int e;
    cout << "Enter base: ";
    cin >> b;
    cout << "Enter exponent (integer): ";
    cin >> e;
    double res = power(b, e);
    cout << b << "^" << e << " = " << res << endl;
    return 0;
}
```

2. Exponential Function with O(N) using Divide and Conquer Approach

The Divide and Conquer method works by splitting the problem into smaller subproblems. In the basic version, we reduce the exponent by 1 in each recursive call, similar to the iterative method but using recursion.

Algorithm:

If n=0n = 0n=0, return 1.

Else, return x × power(x,n−1).

Time Complexity:

O(n) (same as naive but implemented recursively).

Space Complexity:

O(n) due to recursion stack.

Use Case:

- Useful for teaching recursion.
- Not efficient for large n.

Code

```cpp
#include <bits/stdc++.h>
using namespace std;

// Naive divide-and-conquer that leads to O(n) time:
// T(n) = 2T(n/2) + O(1) -> O(n)
long long powerDAC_ON(long long b, long long n) {
    if (n == 0) return 1;
    if (n == 1) return b;
    if (n % 2 == 0) {
        long long half = powerDAC_ON(b, n / 2);     // recompute twice
        long long half2 = powerDAC_ON(b, n / 2);    // causes O(n) overall
        return half * half2;
    } else {
        return b * powerDAC_ON(b, n - 1);
    }
}

int main() {
    long long b, n;
    cin >> b >> n;
    cout << powerDAC_ON(b, n) << "\n";
    return 0;
}
```

3. Exponential Function with O(logN) using Divide and Conquer Approach

This optimized method reduces the number of multiplications by halving the exponent at each step.

Algorithm:

- If n=0n = 0n=0, return
- Compute half = power(x, n // 2).
- If n is even, return half × half.
- If n is odd, return x × half × half.

Time Complexity:

O(log n) (exponent is halved each recursion).

Space Complexity:

O(log n) due to recursion stack.

Use Case:

Ideal for large exponents where performance is critical.

Code:

```cpp
#include <bits/stdc++.h>
using namespace std;

// Efficient exponentiation (binary exponentiation)
long long powerDAC_OlogN(long long b, long long n) {
    if (n == 0) return 1;
    long long half = powerDAC_OlogN(b, n / 2);
    long long sq = half * half;
    if (n % 2 == 0) return sq;
    return sq * b;
}

int main() {
    long long b, n;
    cin >> b >> n;
    cout << powerDAC_OlogN(b, n) << "\n";
    return 0;
}
```

Observation

This experiment highlights how different algorithmic strategies—Iterative, Linear Recursion, and Binary Exponentiation—drastically impact the performance of a seemingly simple mathematical operation (x^n).

Conclusion

1. **The Power of Divide and Conquer**
   The experiment demonstrates that not all Divide and Conquer (D&C) approaches are created equal.
   - A Linear D&C approach (reducing n by 1 each time) provides no speed advantage over a simple loop and actually performs worse due to O(n) stack space usage.
   - A Logarithmic D&C approach (halving n each time) is the true strength of this paradigm. By calculating (x^{n/2})^2, we avoid redundant calculations, making it the standard for high-performance computing.

2. **Efficiency Gains**
   - As n grows, the difference becomes massive. For n = 1,000,000:
      - The Iterative and Linear D&C methods require 1,000,000 multiplications.
      - The Logarithmic D&C method requires only about 20 multiplications.

3. **Practical Application**
   While the iterative approach is fine for basic applications, the O(log n) Divide and Conquer method (often called Binary Exponentiation) is essential in fields like cryptography (e.g., RSA encryption) and competitive programming, where working with massive exponents and modular arithmetic is common.

Merge Sort

Theory:

- Merge Sort is a divide and conquer sorting algorithm that divides the array into smaller subarrays, sorts them, and then merges them back to form a sorted array.
- It works by recursively splitting the array into two halves until each subarray contains only one element.
- Then, it merges these subarrays in a sorted manner to produce the final sorted array.
- The merging process compares elements from both subarrays and places them in the correct order.
- Merge Sort is a stable sorting algorithm, meaning it preserves the order of equal elements.
- It performs consistently well regardless of the initial order of elements.
- However, it requires additional memory for merging, unlike in-place sorting algorithms.

Code:

```
#include <iostream>
using namespace std;
void merge(int arr[], int left, int mid, int right){
    int s1 = mid - left;
    int s2 = right - mid + 1;
    int larr[s1];
    int rarr[s2];
    for (int i = 0; i < s1; i++){
        larr[i] = arr[left + i];
    }
    for (int i = 0; i < s2; i++){
        rarr[i] = arr[mid + i];
    }

    int i = 0;
    int j = 0;
    int k = left;
    while (i < s1 && j < s2){
        if (larr[i] < rarr[j])
        {
            arr[k] = larr[i];
            i++;
        }
```

```
            else{
                arr[k] = rarr[j];
                j++;
            }
            k++;
        }
        while (i < s1){
            arr[k] = larr[i];
            i++;
            k++;
        }
        while (j < s2){
            arr[k] = rarr[j];
            j++;
            k++;
        }
    }
    void merge_sort(int arr[], int left, int right){
        if (left < right){
            int mid = left + (right - left) / 2;
            merge_sort(arr, left, mid);
            merge_sort(arr, mid + 1, right);
            merge(arr, left, mid + 1, right);
        }
    }

    void printArray(int a[], int size){
        for (int i = 0; i < size; i++){
            cout << a[i];
        }
        cout << endl;
```

```
        cout << endl;
    }
    int main(){
        cout << "Enter size of array: ";
        int size;
        cin >> size;

        int array[size];
        for (int ind = 0; ind < size; ind++){
            cout << "Enter array element at index " << ind <<": ";
            cin >> array[ind];
        }

        cout << "Original: ";
        printArray(array, size);
        merge_sort(array, 0, size - 1);
        cout << "After: " ;
        printArray(array, size);
        return 0;
    }
```

Space complexity O(n) : Merge Sort requires additional memory for:

- Temporary arrays (left, right, and merge)
- Recursive function calls

Since new arrays are created during merging, extra space proportional to input size n is needed → O(n).

Time complexity

Best case time complexity O(n log n) : Even if the array is already sorted, Merge Sort still:

- Divides the array into halves → log n levels
- Merges elements at each level → O(n) work per level Total time = O(n log n).

Worst case time complexity O(n log n) : In the worst case, the same process occurs:

- Recursive splitting → log n levels
- Merging at each level → O(n)

So total operations remain → O(n log n).

Quick Sort

Theory:

- Quick Sort is a divide and conquer sorting algorithm that works by selecting a pivot element and partitioning the array around it.
- It rearranges the elements such that elements smaller than the pivot are placed on the left side, and elements greater than the pivot are placed on the right side.
- After partitioning, the pivot is in its correct sorted position.
- The same process is recursively applied to the left and right subarrays.
- The efficiency of Quick Sort depends on the choice of pivot element.
- It is generally faster than other sorting algorithms in practice due to better cache performance.
- However, it is not a stable sorting algorithm and may degrade in performance if the pivot is poorly chosen.

Code:

```cpp
//Quick Sort
#include <iostream>
#include <vector>
#include <utility> // For swap
using namespace std;

void printVector(const vector<int>& arr) {
    for (int val : arr) {
        cout << val << " ";
    }
    cout << endl;
}

int partition(vector<int>& arr, int low, int high) {
    int pivot = arr[high]; // Choose the last element as the pivot
    int i = (low - 1);     // Index of smaller element

    for (int j = low; j <= high - 1; ++j) {
        // If the current element is smaller than the pivot
        if (arr[j] < pivot) {
            i++; // Increment index of smaller element
            swap(arr[i], arr[j]);
```

```
                swap(arr[i], arr[j]);
            }
        }
        swap(arr[i + 1], arr[high]);
        return (i + 1);
    }

    void quickSort(vector<int>& arr, int low, int high) {
        if (low < high) {
            // pi is the partitioning index, arr[pi] is now at the right place
            int pi = partition(arr, low, high);

            quickSort(arr, low, pi - 1);
            quickSort(arr, pi + 1, high);
        }
    }

    int main() {
        vector<int> data = {10, 7, 8, 9, 1, 5};

        cout << "Unsorted array: ";
        printVector(data);

        quickSort(data, 0, data.size() - 1);

        cout << "Sorted array:   ";
        printVector(data);

        return 0;
    }
```

Output

```
[Running] cd "c:\Users\Mo7am\Downloads\Data-Structure-Algorithms-main\Data-Structu
"c:\Users\Mo7am\Downloads\Data-Structure-Algorithms-main\Data-Structure-Algorithms
Unsorted array: 10 7 8 9 1 5
Sorted array:   1 5 7 8 9 10
```

Space complexity O(log n) : Quick Sort uses **recursion**, and the space is required for the recursion stack. In the average case, the recursion depth is **log n**, so space complexity is **O(log n)**.

Time complexity

Best case time complexity O(n log n) : In the best case, the pivot divides the array into two equal halves each time. Number of levels = log n, and at each level, partitioning takes O(n) → overall O(n log n).

Worst case time complexity O(n²) : In the worst case, the pivot is always the **smallest or largest element** (e.g., already sorted array). This results in highly unbalanced partitions: $T(n) = T(n-1) + O(n)$

Observations

1. Merge Sort (The Reliable Processor)

- **Mechanism:** It recursively divides the array into two equal halves, sorts them, and then merges the sorted halves back together.
- **Consistency:** Its performance is remarkably stable. Whether the data is already sorted, reverse-sorted, or completely random, it always takes the same amount of time.
- **External Sorting:** Because it processes data in predictable chunks, it is the preferred choice for sorting massive datasets that don't fit into a computer's RAM (External Sorting).

2. Quick Sort (The High-Speed Gambler)

- **Mechanism:** It picks a "pivot" element and partitions the array such that elements smaller than the pivot are on the left and larger ones are on the right.
- **Pivot Dependency**: Its efficiency relies heavily on the choice of the pivot. A poor pivot (like the smallest or largest element in an already sorted array) can cause the performance to collapse.
- **In-Place Efficiency:** Unlike Merge Sort, it sorts the data directly within the original array, making it much more memory-efficient.

Conclusion

1. **Memory vs. Speed:** Quick Sort is generally faster in practice for internal sorting (RAM) because it has better "cache locality" and doesn't require the extra $O(n)$ space that Merge Sort needs.
2. **Stability Matters:** If you need to maintain the relative order of duplicate elements (e.g., sorting a list of transactions by date, then by amount), Merge Sort is the correct choice. Quick Sort might shuffle elements with equal values.
3. **Data Structure Choice:** * Merge Sort is highly efficient for Linked Lists because it can be implemented without the $O(n)$ extra space penalty.
    - Quick Sort is the "de-facto" standard for Arrays in most programming libraries (like C++ std::sort or Java's primitive sorts).
4. **The "Worst Case" Risk**: While Merge Sort is guaranteed $O(n \log n)$, Quick Sort carries a risk of $O(n^2)$. This is why modern implementations often use "Randomized Quick Sort" or a hybrid approach to ensure they almost never hit that worst-case scenario.

Marwadi University Marwadi Chandarana Group NAAC A+	**Marwadi University** **Faculty of Engineering and Technology** **Department of Information and Communication Technology**	
Design and Analysis of Algorithm (01CT0512)	**Aim: Implementing The Karatsuba Algorithm**	
Experiment No: 5	**Date: 9/4/2026**	**Enrolment No: 92301733059**

Theory:

- Karatsuba algorithm is an efficient divide and conquer algorithm used for fast multiplication of large numbers.
- It improves over the traditional multiplication method by reducing the number of multiplications required.
- The numbers are split into two halves (high and low parts) based on their digits.
- Instead of performing four multiplications as in the naive approach, Karatsuba uses only three multiplications, which reduces computation time.
- The algorithm recursively applies the same method to smaller parts until the base case is reached.
- The key idea is based on the formula:

$$x \cdot y = 10^{n} \cdot ac + 10^{n/2} \cdot ((a + b)(c + d) - ac - bd) + bd$$

where the numbers are split as:

$x = a \cdot 10^{n/2} + b$ and $y = c \cdot 10^{n/2} + d$.

- Karatsuba significantly reduces the time complexity compared to classical multiplication.
- It is widely used in applications involving large integer arithmetic, such as cryptography.

Code

```
// karatsuba
#include <iostream>
#include <string> // It's good practice to include <string> for to_string
#include <cmath>
#include <algorithm> // It's good practice to include <algorithm> for max

using namespace std;

long long karatsuba(long long x, long long y){
    // Base case for recursion
    if (x < 10 || y < 10){
        return x * y;
    }

```

```
    // Calculate the size of the numbers
    int m = max(to_string(x).length(), to_string(y).length());
    int m2 = m / 2;
    // Split the numbers into two halves
    long long power = pow(10, m2);
    long long a = x / power;
    long long b = x % power;
    long long c = y / power;
    long long d = y % power;
    // Recursive steps
    long long ac = karatsuba(a, c);                // z2
    long long bd = karatsuba(b, d);                // z0
    long long ab_cd = karatsuba((a + b), (c + d)); // z1

    // Combine the results using the Karatsuba formula
    // The error was here: it should be + bd at the end, not - bd
    return (ac * pow(10, m2 * 2)) + ((ab_cd - ac - bd) * power) + bd;
}
int main(){
    long long x =1234, y=5678;
    // cout << "Enter first number: ";
    // cin >> x;
    // cout << "Enter second number: ";
    // cin >> y;
    long long result = karatsuba(x, y);
    cout << x << " * " << y << " = " << result << endl;
    cout << "Correct answer: " << x * y << endl; // For verification
    return 0;
}
```

Output

```
[Running] cd "c:\Users\Mo7am\Downloads\Data-Structure-Algorit
"c:\Users\Mo7am\Downloads\Data-Structure-Algorithms-main\Data
1234 * 5678 = 7006652
Correct answer: 7006652
```

Space complexity O(log n) : Karatsuba uses recursion, and the depth of recursion is proportional to log n (since the number is divided into halves at each step). Thus, extra space is required for the recursion stack → O(log n).

Time complexity

Best case time complexity: $O(n^{\log_2 3}) \approx O(n^{1.58})$, Karatsuba reduces the multiplication problem into 3 recursive subproblems of size n/2:

$T(n) = 3T(n/2) + O(n)$

Solving this recurrence gives:

$T(n) = O(n\log 2\ 3) \approx O(n1.58)$ This applies when recursion is balanced → best case.

Worst case time complexity: $O(n^{\log_2 3}) \approx O(n^{1.58})$

Justification: Even in the worst case, the algorithm still performs:

- 3 recursive multiplications
- Linear work for additions and shifts

So the recurrence remains the same → $O(n^{1.58})$.

Conclusion

The Karatsuba algorithm represents a fundamental shift in computational arithmetic, proving that the "obvious" $O(n^2)$ approach to multiplication is not the most efficient. By cleverly applying a **divide-and-conquer** strategy and algebraic manipulation, the algorithm reduces the number of recursive multiplications from four to three.

Key Takeaways

- **Algorithmic Efficiency:** The transition from $O(n^2)$ to approximately $O(n^{1.58})$ significantly lowers the computational floor for large-scale arithmetic. While the overhead of recursion and addition makes it slower for small numbers, it becomes exponentially superior as the number of digits (n) increases.
- **Space-Time Trade-off:** The algorithm achieves its speed at the cost of $O(\log n)$ space due to the recursion stack. This is a manageable trade-off in modern computing environments where memory is often more abundant than raw processing cycles.
- **Real-World Application:** Karatsuba is not merely a theoretical exercise; it is the backbone of modern **cryptographic systems** (like RSA) and big-integer libraries in languages like Python and Java. Without such sub-quadratic algorithms, secure data encryption would be significantly more resource-intensive.

Final Assessment: The Karatsuba algorithm serves as a landmark in computer science, illustrating that mathematical insight can overcome hardware limitations. It remains one of the most elegant examples of how reducing sub-problems in a recursive structure can fundamentally alter the complexity of an operation.

Marwadi University Marwadi Chandarana Group NAAC A+	**Marwadi University** **Faculty of Engineering and Technology** **Department of Information and Communication Technology**	
Design and Analysis of Algorithm (01CT0512)	**Aim: Implementing The Knapsack Algorithm**	
Experiment No: 6	**Date: 9/4/2026**	**Enrolment No: 92301733059**

Fractional Knapsack using Greedy

Theory:

- The Fractional Knapsack problem is an optimization problem where we aim to maximize the total value of items placed in a knapsack with a limited capacity.
- Each item has a weight and a value, and we can take fractions of items unlike the 0/1 Knapsack problem.
- The Greedy approach selects items based on the highest value-to-weight ratio (value/weight).
- Items are sorted in decreasing order of their value-to-weight ratio.
- The algorithm picks items one by one:
    - If the whole item can fit, it is fully included.
    - Otherwise, only the fraction that fits into the remaining capacity is taken.
    - This process continues until the knapsack is full.
    - The Greedy strategy works optimally for the Fractional Knapsack problem because taking the locally best choice (highest ratio) leads to a globally optimal solution.
    - However, this approach does not work for 0/1 Knapsack, where items must be taken completely or not at all.

Code:

```cpp
//knapsack problem
#include <iostream>
#include <vector>
#include <algorithm>
using namespace std;

/**
 * @brief Solves the 0/1 Knapsack problem using dynamic programming.
 * * This function determines the maximum value that can be obtained by selecting
 * items from a given set, subject to a total weight constraint. Each item
 * can either be included in the knapsack or not (0/1 property).
 *
 * @param weights A constant reference to a vector of integers representing the weights of the items.
 * @param values A constant reference to a vector of integers representing the values of the items.
 * @param capacity The maximum weight capacity of the knapsack.
 * @return The maximum total value that can be carried in the knapsack.
 */
```

```
18  int knapsack_01(const vector<int>& weights, const vector<int>& values, int capacity) {
19      // Get the number of items.
20      int n = values.size();
21
22      // Create a 2D DP table.
23      // dp[i][w] will store the maximum value that can be achieved with a capacity of 'w'
24      // using only the first 'i' items.
25      // The table size is (n+1) x (capacity+1) to handle base cases where i=0 or w=0.
26      vector<vector<int>> dp(n + 1, vector<int>(capacity + 1, 0));
27
28      // Iterate through each item
29      for (int i = 1; i <= n; ++i) {
30          // Iterate through each possible weight capacity from 1 to the max capacity
31          for (int w = 1; w <= capacity; ++w) {
32              // The current item's weight and value. We use i-1 because vector is 0-indexed.
33              int current_weight = weights[i - 1];
34              int current_value = values[i - 1];
35
36              // If the current item's weight is greater than the current capacity 'w',
37              // we cannot include it. So, the maximum value is the same as the value
38              // without this item (from the row above).
39              if (current_weight > w) {
40                  dp[i][w] = dp[i - 1][w];
41              } else {
42                  // If the item can fit, we have two choices:
43                  // 1. Don't include the item: The value is dp[i-1][w].
44                  // 2. Include the item: The value is the current item's value plus the value
45                  //    of the remaining items with the remaining capacity.
46                  int value_without_item = dp[i - 1][w];
47                  int value_with_item = current_value + dp[i - 1][w - current_weight];
48
49                  // We choose the option that gives the maximum value.
50                  dp[i][w] = max(value_without_item, value_with_item);
51              }
52          }
53      }
54
55      // The bottom-right cell of the table contains the maximum value for all 'n' items
56      // with the full 'capacity'.
57      return dp[n][capacity];
58  }
```

```
// --- Example Usage ---
int main() {
    // Define the items with their respective values and weights
    vector<int> item_values = {60, 100, 120};
    vector<int> item_weights = {10, 20, 30};
    int knapsack_capacity = 50;

    // Call the function to find the maximum value
    int max_value = knapsack_01(item_weights, item_values, knapsack_capacity);

    // Print the results
    cout << "Item Values: {60, 100, 120}" << endl;
    cout << "Item Weights: {10, 20, 30}" << endl;
    cout << "Knapsack Capacity: " << knapsack_capacity << endl;
    cout << "-----------------------------" << endl;
    // The optimal solution is to choose the items with weights 20 and 30 (total 50),
    // giving a total value of 100 + 120 = 220.
    cout << "Maximum value in knapsack: " << max_value <<endl;

    return 0;
}
```

Output

```
"c:\Users\Mo7am\Downloads\Data-Structure-Algorithms-main\Data-Structure
Item Values: {60, 100, 120}
Item Weights: {10, 20, 30}
Knapsack Capacity: 50
-----------------------------
Maximum value in knapsack: 220
```

Space complexity O(1) : The algorithm uses only a few extra variables (total_value, capacity). Sorting can be done in-place, so no significant extra memory is required apart from input storage → constant auxiliary space.

Time complexity:

Best case time complexity O(n log n) : The dominant step is sorting the items based on value/weight ratio, which takes O(n log n) time. Even if the knapsack gets filled early, sorting is still required → overall O(n log n).

Worst case time complexity O(n log n) : In the worst case:

- All items are processed after sorting
- Sorting takes O(n log n)
- Traversing items takes O(n)

Total complexity = O(n log n) (sorting dominates).

knapsack using greedy

```cpp
#include <iostream>
#include <vector>
using namespace std;

int knapsack(int W, vector<int>& wt, vector<int>& val, int n) {
    vector<vector<int>> dp(n + 1, vector<int>(W + 1, 0));

    for (int i = 1; i <= n; i++) {
        for (int w = 0; w <= W; w++) {
            if (wt[i - 1] <= w)
                dp[i][w] = max(dp[i - 1][w], dp[i - 1][w - wt[i - 1]] + val[i - 1]);
            else
                dp[i][w] = dp[i - 1][w];
        }
    }

    return dp[n][W];
}

int main() {
    vector<int> val = {60, 100, 120};
    vector<int> wt = {10, 20, 30};
    int W = 50;
    int n = val.size();
    cout << "Maximum value in knapsack: " << knapsack(W, wt, val, n) << endl;
    return 0;
}
```

```
[Running] cd "c:\Users\Mo7am\Downloads\Data-Structure-Algorithms-main\Data-Structu
"c:\Users\Mo7am\Downloads\Data-Structure-Algorithms-main\Data-Structure-Algorithm
Maximum value in knapsack: 220
```

Conclusion

1. The Greedy Approach

The Greedy method works by making the locally optimal choice at each step—specifically, by picking items with the highest **value-to-weight ratio** first.

- **Best For:** The **Fractional Knapsack** problem. Since you can take fractions of an item, the greedy strategy always yields the maximum possible value.
- **Efficiency:** It is highly efficient with a time complexity of O(n log n) due to the initial sorting of items.
- **Limitation:** It **fails** for the **0/1 Knapsack** problem. Because you cannot take a partial item, a greedy choice early on might "waste" space that could have been better filled by a different combination of items.

2. The Dynamic Programming Approach

Dynamic Programming solves the problem by breaking it down into smaller sub-problems and storing the results in a table (memoization) to avoid redundant calculations.

- **Best For:** The **0/1 Knapsack** problem. It explores all possible combinations of items to ensure the global optimum is found.
- **Efficiency:** It has a time complexity of O(n W), where n is the number of items and W is the maximum capacity of the knapsack.
- **Limitation:** It is more "expensive" in terms of memory and time than the Greedy approach, especially if the knapsack capacity W is very large.

Marwadi University Marwadi Chandarana Group NAAC A+	**Marwadi University** **Faculty of Engineering and Technology** **Department of Information and Communication Technology**	
Design and Analysis of Algorithm (01CT0512)	**Aim: Implementing the Job Scheduling**	
Experiment No: 7	**Date: 9/4/2026**	**Enrolment No: 92301733059**

Job Scheduling

Theory:

- Job Scheduling is a problem where we have a set of jobs, each with a deadline and a profit.
- Each job takes one unit of time, and only one job can be scheduled at a time.
- The objective is to maximize total profit by scheduling jobs within their deadlines.
- The Greedy approach selects jobs based on maximum profit first.
- Jobs are sorted in descending order of profit.
- Each job is assigned to the latest available time slot before its deadline.
- If no slot is available before the deadline, the job is skipped.
- This process continues until all jobs are considered.
- The Greedy strategy works because choosing the highest profit job first helps achieve a globally optimal solution.

Code:

```
#include <iostream>
#include <vector>
#include <algorithm>

using namespace std;

// Structure to represent a Job
struct Job {
    char id;      // Job ID
    int dead;     // Deadline of job
    int profit;   // Profit if job is over before or on deadline
};

// Comparator function to sort jobs in descending order of profit
bool comparison(Job a, Job b) {
    return (a.profit > b.profit);
}

void printJobScheduling(Job arr[], int n) {
    // 1. Sort all jobs according to decreasing order of profit
    sort(arr, arr + n, comparison);

```

```
    vector<int> result(n); // To store result (Sequence of jobs)
    vector<bool> slot(n, false); // To keep track of free time slots
    int totalProfit = 0;
    int countJobs = 0;

    // 2. Iterate through all given jobs
    for (int i = 0; i < n; i++) {
        // Find a free slot for this job (starting from the last possible slot)
        for (int j = min(n, arr[i].dead) - 1; j >= 0; j--) {
            if (slot[j] == false) {
                result[j] = i;  // Add this job to result
                slot[j] = true; // Make this slot occupied
                totalProfit += arr[i].profit;
                countJobs++;
                break;
            }
        }
    }
    // Print the sequence
    cout << "Following is the maximum profit sequence of jobs: ";
    for (int i = 0; i < n; i++) {
        if (slot[i]) {
            cout << arr[result[i]].id << " ";
        }
    }
    cout << "\nTotal Jobs Scheduled: " << countJobs;
    cout << "\nTotal Profit: " << totalProfit << endl;
}
int main() {
```

```
int main() {
    // Example set of jobs
    Job arr[] = { {'A', 2, 100}, {'B', 1, 19}, {'C', 2, 27},
                  {'D', 1, 25}, {'E', 3, 15} };
    int n = sizeof(arr) / sizeof(arr[0]);

    printJobScheduling(arr, n);
    return 0;
}
```

Output

```
"c:\Users\Mo7am\Downloads\Data-Structure-Algorithms-main\Data-Str
Following is the maximum profit sequence of jobs: C A E
Total Jobs Scheduled: 3
Total Profit: 142
```

Time Complexity

- **Best Case: O(n log n)**
  This occurs if the number of jobs is very small or if the deadlines are structured such that we find a slot immediately for every job. However, the initial **sorting** of n jobs by profit still takes O(n log n).
- **Worst Case: O(n^2)**
  In the standard greedy implementation (like the code above), for every job, we might have to search through all n time slots to find an empty one. If we have n jobs and the maximum deadline is n, the nested loops result in O(n^2).
  Note: This can be optimized to O(n log n) using a **Disjoint Set Union (DSU)** data structure to find available slots faster.

Space Complexity

- **Auxiliary Space: O(n)**
  Unlike the Fractional Knapsack analysis you shared, Job Scheduling usually requires $O(n)$ extra space. This is because we must maintain a **boolean array or vector** to track which time slots are occupied and a **result array** to store the sequence of jobs.
- **In-place Sorting:** While sorting can be done in O(1) auxiliary space (using Heapsort), the requirement to track n individual time slots makes the overall auxiliary space O(n).

Conclusion

The **Job Scheduling** algorithm is a greedy powerhouse because it ensures the highest profit jobs are prioritized. While its basic implementation has a quadratic time complexity ($O(n^2)$), its logic is straightforward:

1. **Prioritize Profit:** Always look at the most "expensive" job first.
2. **Delay Execution:** By scheduling a job at the latest possible moment (its deadline), you leave the earlier slots open for jobs that *must* be done early.
3. **Global Optimum:** For this specific version of the problem (where each job takes 1 unit of time), the Greedy approach is guaranteed to find the best possible total profit.

Marwadi University Marwadi Chandarana Group NAAC A+	**Marwadi University** **Faculty of Engineering and Technology** **Department of Information and Communication Technology**	
Design and Analysis of Algorithm (01CT0512)	**Aim: Implementing the Activity Selection**	
Experiment No: 8	**Date: 9/4/2026**	**Enrolment No: 92301733059**

the Activity Selection

Theory:

- The Activity Selection problem involves selecting the maximum number of activities that can be performed by a single person, given their start and finish times.
- The condition is that no two selected activities should overlap in time.
- The Greedy approach selects activities based on the earliest finishing time.
- First, all activities are sorted in increasing order of their finish times.
- The first activity (with the earliest finish time) is always selected.
- Then, the next activity is chosen such that its start time is greater than or equal to the finish time of the last selected activity.
- This process continues until all activities are considered.
- The Greedy strategy works because selecting the activity that finishes earliest leaves more room for selecting remaining activities.
- This approach guarantees the maximum number of non-overlapping activities.

Code:

```
#include <iostream>
#include <vector>
#include <algorithm>

using namespace std;

// Structure to hold Activity details
struct Activity {
    int id;
    int start;
    int finish;
};

// Comparator: Sort activities based on finish time
bool activityCompare(Activity s1, Activity s2) {
    return (s1.finish < s2.finish);
}
```

```
}

void selectActivities(vector<Activity>& activities) {
    // 1. Sort activities by finish time
    sort(activities.begin(), activities.end(), activityCompare);
    cout << "Selected Activities: ";
    // 2. The first activity is always selected
    int i = 0;
    cout << "A" << activities[i].id << " ";

    // 3. For the rest of the activities, check if start time
    // is greater than or equal to the finish time of the last selected activity
    for (int j = 1; j < activities.size(); j++) {
        if (activities[j].start >= activities[i].finish) {
            cout << "A" << activities[j].id << " ";
            i = j; // Update the index of the last selected activity
        }
    }
    cout << endl;
}
int main() {
    vector<Activity> activities = {
        {1, 1, 4}, {2, 3, 5}, {3, 0, 6}, {4, 5, 7}, {5, 3, 9}, {6, 5, 9}, {7, 6, 10}, {8, 8, 11}, {9, 8, 12}, {10, 2, 14}, {11, 12, 16}
    };
    selectActivities(activities);
    return 0;
}
```

Output

```
[Running] cd "c:\Users\Mo7am\Downloads\Data-Structure-Algorithms
"c:\Users\Mo7am\Downloads\Data-Structure-Algorithms-main\Data-St
Selected Activities: A1 A4 A8 A11
```

Time Complexity

- **Best Case O(n log n):** Even if the activities are already sorted, we typically perform a check. The O(n log n) comes from the sorting step, which is mandatory to apply the greedy logic.

- **Worst Case O(n log n):** * Sorting: O(n \log n) (using QuickSort or MergeSort).Scanning: After sorting, we iterate through the list once, which takes O(n).Total: O(n log n) + O(n) = O(n log n).

Space Complexity

- **Auxiliary Space O(1):** The algorithm processes the sorted list and uses a few variables (like i and j) to track indices. If the sorting is done in-place, the extra memory used is constant.

Conclusion

The **Activity Selection** problem demonstrates that a "Shortest Task First" or "Earliest Start" approach doesn't always work, but the **"Earliest Finish Time"** greedy choice is optimal.

- **Optimal Substructure:** Once we pick the activity that finishes first, the remaining problem is to solve the activity selection for the remaining time, which is just a smaller version of the original problem.
- **Greedy Choice Property:** Choosing the activity with the earliest finish time leaves the maximum possible time remaining for other activities.
- **Versatility:** This logic is widely used in real-world scheduling, CPU task management, and classroom/meeting room allocations.

Marwadi University Marwadi Chandarana Group NAAC A+	**Marwadi University** **Faculty of Engineering and Technology** **Department of Information and Communication Technology**	
Design and Analysis of Algorithm (01CT0512)	**Aim: Implementing the Longest Common Subsequence**	
Experiment No:9	**Date: 9/4/2026**	**Enrolment No: 92301733059**

Longest Common Subsequence

Theory:

Longest Common Subsequence (LCS):

- The Longest Common Subsequence (LCS) problem is used to find the longest sequence that is common to two sequences (strings or arrays).
- A subsequence is a sequence derived from another sequence by deleting some or no elements without changing the order of the remaining elements.
- The goal is to determine the maximum length subsequence that appears in both sequences.
- LCS uses the Dynamic Programming approach to avoid redundant computations.
- A 2D table (matrix) is created where rows represent one string and columns represent the other string.
- If the characters match, the value is incremented by 1 from the diagonal cell.
- If the characters do not match, the maximum value from the left or top cell is taken.
- The final answer is obtained from the bottom-right cell of the table.
- LCS is widely used in applications such as DNA sequence analysis, file comparison, and version control systems.

Code:

```
#include <iostream>
#include <string>
#include <vector>
#include <algorithm>

// Use the standard namespace to avoid prefixing std::
using namespace std;

/**
 * @brief Calculates the length of the Longest Common Subsequence of two strings.
 * This function uses a dynamic programming approach to find the length of the LCS.
 * @param s1 The first string.
 * @param s2 The second string.
 * @return The length of the longest common subsequence.
 */
int findLCSLength(const string& s1, const string& s2) {
    int m = s1.length();
    int n = s2.length();
```

```
    int n = s2.length();

    // Create a DP table (matrix) to store the lengths of LCSs.
    // dp[i][j] will store the length of LCS of s1[0..i-1] and s2[0..j-1].
    // Initialize with all zeros.
    vector<vector<int>> dp(m + 1, vector<int>(n + 1, 0));

    // Build the dp table in a bottom-up manner.
    for (int i = 1; i <= m; ++i) {
        for (int j = 1; j <= n; ++j) {
            // If the characters at the current positions match,
            // the LCS is one character longer than the LCS of the strings without these characters.
            if (s1[i - 1] == s2[j - 1]) {
                dp[i][j] = 1 + dp[i - 1][j - 1];
            }
            // If the characters do not match, the LCS is the longer of the two possibilities:
            // 1. LCS of s1[0..i-2] and s2[0..j-1]
            // 2. LCS of s1[0..i-1] and s2[0..j-2]
            else {
                dp[i][j] = max(dp[i - 1][j], dp[i][j - 1]);
            }
        }
    }

    // The value at the bottom-right corner of the table is the length of the LCS
    // for the entire strings.
    return dp[m][n];
}


// --- Example Usage ---
int main() {
    string str1 = "AGGTAB";
    string str2 = "GXTXAYB";

    cout << "String 1: " << str1 << endl;
    cout << "String 2: " << str2 << endl;
    cout << "-----------------------------" << endl;

    cout << "Length of the Longest Common Subsequence is: " << length << endl;
    // For "AGGTAB" and "GXTXAYB", the LCS is "GTAB", so the length is 4.

    return 0;
}
```

Code view is read-only. Switch to the editor.

Output

```
longestCommonSubsequence && "c:\Users\Mo7am\Downloads\Data-Structure-
String 1: AGGTAB
String 2: GXTXAYB
-----------------------------
Length of the Longest Common Subsequence is: 4

[Done] exited with code=0 in 3.225 seconds
```

Time Complexity

- **Worst Case:** O(m.n) We use two nested loops to fill a table of size (m+1) times (n+1) , where m and n are the lengths of the two strings.Best Case: O(m . n) Even if the strings are identical or completely different, the DP approach still evaluates every subproblem in the grid to guarantee the result.

Space Complexity

- **Auxiliary Space: O(m.n):** A 2D array is required to store the intermediate results for backtracking.

Conclusion

The **Longest Common Subsequence** problem highlights why **Dynamic Programming** is essential for problems involving overlapping subproblems and optimal substructure.

- **Why Greedy Fails:** A greedy approach might match the first available character (e.g., 'A' in "ABC" and "ACB"), potentially missing a longer sequence that starts later. DP avoids this by looking at all possibilities.
- **Optimal Substructure:** The solution to the problem LCS(S1[1..i], S2[1..j]) depends on the solutions to smaller prefixes.
- **Applications:** LCS is the backbone of the **diff** command in Unix/Linux, version control systems like **Git** (for merging changes), and **Bioinformatics** (comparing DNA sequences).

Space Complexity

- **Auxiliary Space O(1):** The algorithm processes the sorted list and uses a few variables (like i and j) to track indices. If the sorting is done in-place, the extra memory used is constant.

Conclusion

The **Activity Selection** problem demonstrates that a "Shortest Task First" or "Earliest Start" approach doesn't always work, but the **"Earliest Finish Time"** greedy choice is optimal.

- **Optimal Substructure:** Once we pick the activity that finishes first, the remaining problem is to solve the activity selection for the remaining time, which is just a smaller version of the original problem.
- **Greedy Choice Property:** Choosing the activity with the earliest finish time leaves the maximum possible time remaining for other activities.
- **Versatility:** This logic is widely used in real-world scheduling, CPU task management, and classroom/meeting room allocations.

Marwadi University Marwadi Chandarana Group NAAC A+	**Marwadi University** **Faculty of Engineering and Technology** **Department of Information and Communication Technology**	
Design and Analysis of Algorithm (01CT0512)	**Aim: Implementing the Navie Approach – String Matching**	
Experiment No:10	**Date: 9/4/2026**	**Enrolment No: 92301733059**

Naive Approach – String Matching

Theory:

- The Naive String Matching algorithm is the simplest method used to find a pattern string within a text string.
- It works by checking the pattern at every possible position in the text.
- For each position, it compares the pattern with the corresponding substring of the text character by character.
- If all characters match, the pattern is found at that position.
- If a mismatch occurs, the algorithm shifts the pattern by one position to the right and repeats the process.
- This continues until the pattern is found or all positions in the text are checked.
- It does not use any preprocessing or optimization techniques.
- Although simple to implement, it is inefficient for large texts due to repeated comparisons.

Code:

```
#include <iostream>
#include <string>
#include <vector>

using namespace std;

void naiveSearch(string text, string pattern) {
    int n = text.length();
    int m = pattern.length();

    cout << "Searching for '" << pattern << "' in '" << text << "'..." << endl;
    bool found = false;

    // Slide the pattern over text one by one
    for (int i = 0; i <= n - m; i++) {
        int j;
```

```
            // For current index i, check for pattern match
            for (j = 0; j < m; j++) {
                if (text[i + j] != pattern[j]) {
                    break; // Mismatch found, break internal loop
                }
            }

            // If j reached m, then pattern found at current index i
            if (j == m) {
                cout << "Pattern found at index " << i << endl;
                found = true;
            }
        }

        if (!found) {
            cout << "Pattern not found in the given text." << endl;
        }
    }

    int main() {
        string text = "AABAACAADAABAAABAA";
        string pattern = "AABA";

        naiveSearch(text, pattern);

        return 0;
    }
```

Output

```
"c:\Users\Mo7am\Downloads\Data-Structure-Algorithms-ma
Searching for 'AABA' in 'AABAACAADAABAAABAA'...
Pattern found at index 0
Pattern found at index 9
Pattern found at index 13
```

Time Complexity

- **Best Case: O(n)**
  This occurs when the first character of the pattern does not match any character in the text (except where the pattern actually exists). The inner loop breaks almost immediately every time.

- **Worst Case: O(m (n - m + 1))**
  This occurs when all characters of the text and pattern are the same, or only the last character of the pattern is different (e.g., Text: "AAAAAAAAAAAAAAAAB", Pattern: "AAAB"). The algorithm performs m comparisons for almost every shift.

Space Complexity

- **Auxiliary Space:** O(1) The algorithm only uses a few integer variables (i, j, n, m) to keep track of indices. No extra data structures are required regardless of the input size.

Conclusion

The Naive String Matching algorithm serves as the foundational baseline for all string-searching problems. While it is not the most efficient, it is the most intuitive.

- **No Preprocessing: Unlike advanced algorithms like KMP (Knuth-Morris-Pratt) or Rabin-Karp, the Naive approach requires zero preprocessing of the pattern or text.**
- **Inefficiency with Repetition: Its main drawback is that it "forgets" what it has already seen. If a mismatch occurs at the end of a long pattern, it slides forward only one position and starts the comparison from scratch.**
- **Usage: It is perfectly suitable for short texts or applications where simplicity is more important than raw speed. For large-scale data (like searching through DNA sequences or large documents), more optimized algorithms are preferred.**

Rabin Karp-string Matching

Theory:

- Rabin-Karp is an efficient string matching algorithm that uses hashing to find a pattern in a text.
- Instead of comparing characters one by one, it compares the hash value of the pattern with the hash value of substrings of the text.
- First, the hash value of the pattern is computed.
- Then, a hash value is calculated for the first window (substring) of the text with the same length as the pattern.
- The algorithm then slides the window one position at a time and updates the hash value using a rolling hash technique.
- If the hash values match, a character-by-character comparison is performed to confirm the match (to avoid false positives due to hash collisions).
- If the hash values do not match, the window is shifted, and the hash is updated efficiently without recomputing from scratch.
- This process continues until the entire text is scanned.
- Rabin-Karp is particularly useful for multiple pattern matching and large text searches.

Code:

```
#include <iostream>
#include <string>
#include <vector>
#include <cmath>

using namespace std;

#define D 256 // Number of characters in the input alphabet
#define Q 101 // A prime number for modular arithmetic

void rabinKarpSearch(string text, string pattern) {
    int n = text.length();
    int m = pattern.length();
```

```
        int p = 0; // Hash value for pattern
        int t = 0; // Hash value for text window
        int h = 1;

        // The value of h would be "pow(D, m-1) % Q"
        for (int i = 0; i < m - 1; i++)
            h = (h * D) % Q;

        // Calculate initial hash value of pattern and first window of text
        for (int i = 0; i < m; i++) {
            p = (D * p + pattern[i]) % Q;
            t = (D * t + text[i]) % Q;
        }

        // Slide the pattern over text one by one
        for (int i = 0; i <= n - m; i++) {
            // If hash values match, check characters one by one
            if (p == t) {
                int j;
                for (j = 0; j < m; j++) {
                    if (text[i + j] != pattern[j])
                        break;
                }
                if (j == m)
                    cout << "Pattern found at index " << i << endl;
            }

            // Calculate hash value for next window of text
            if (i < n - m) {
                t = (D * (t - text[i] * h) + text[i + m]) % Q;

                // We might get negative value of t, converting it to positive
```


```
                // We might get negative value of t, converting it to positive
```


```
                if (t < 0)
                    t = (t + Q);
            }
        }
    }

    int main() {
        string text = "GEEKS FOR GEEKS";
        string pattern = "GEEK";
        rabinKarpSearch(text, pattern);
        return 0;
    }
```

Output

```
[Running] cd "c:\Users\Mo7am\Downloads\Data-Structure-Algorithms-main\Data
"c:\Users\Mo7am\Downloads\Data-Structure-Algorithms-main\Data-Structure-Al
Pattern found at index 0
Pattern found at index 10
```

Time Complexity

- **Best and Average Case: O(n + m)**
  When the hash function is effective and results in few "spurious hits" (hash matches but string doesn't), we only spend linear time sliding through the text.

- **Worst Case: O(n . m)**
  This happens if the hash function results in a collision at every single shift (e.g., searching for "AAAA" in "AAAAAAAAAAAAAAAA"), forcing the algorithm to perform character comparisons at every step, just like the Naive approach.

Space Complexity

- **Auxiliary Space: O(1)** The algorithm only stores a few integer hash values and constants. It processes the text in a single pass without needing extra data structures like tables or trees.

Conclusion

The Rabin-Karp algorithm is a powerful example of using hashing to speed up search operations. It transitions the problem from character matching to numerical comparison.

- Rolling Hash Advantage: The "rolling" nature of the hash allows us to calculate the hash of the next window in O(1) time by removing the leading character and adding the trailing one, rather than re-hashing the entire window.
- Multiple Pattern Search: One of Rabin-Karp's biggest strengths is its ability to search for *multiple* patterns simultaneously. By storing hashes of different patterns in a Hash Table (Bloom Filter), you can check the text once for all of them.
- Spurious Hits: The primary weakness is the possibility of different strings having the same hash value. Choosing a large prime number (Q) and a suitable base (D) is critical to keeping the algorithm efficient.

Marwadi University Marwadi Chandarana Group NAAC A+	**Marwadi University** **Faculty of Engineering and Technology** **Department of Information and Communication Technology**	
Design and Analysis of Algorithm (01CT0512)	**Aim: Implementing the DFS and BFS**	
Experiment No:12	**Date: 9/4/2026**	**Enrolment No: 92301733059**

the DFS and BFS

Theory:

- Rabin-Karp is an efficient string matching algorithm that uses hashing to find a pattern in a text.
- Instead of comparing characters one by one, it compares the hash value of the pattern with the hash value of substrings of the text.
- First, the hash value of the pattern is computed.
- Then, a hash value is calculated for the first window (substring) of the text with the same length as the pattern.
- The algorithm then slides the window one position at a time and updates the hash value using a rolling hash technique.
- If the hash values match, a character-by-character comparison is performed to confirm the match (to avoid false positives due to hash collisions).
- If the hash values do not match, the window is shifted, and the hash is updated efficiently without recomputing from scratch.
- This process continues until the entire text is scanned.
- Rabin-Karp is particularly useful for multiple pattern matching and large text searches.

Code:

```
#include <iostream>
#include <vector>
#include <list>
#include <queue>
#include <stack>

using namespace std;

class Graph {
    int V; // Number of vertices
    list<int> *adj; // Adjacency list

```

```
    public:
        Graph(int V) {
            this->V = V;
            adj = new list<int>[V];
        }

        void addEdge(int v, int w) {
            adj[v].push_back(w); // Add w to v's list.
        }

        // BFS Traversal from a given source s
        void BFS(int s) {
            vector<bool> visited(V, false);
            queue<int> q;

            visited[s] = true;
            q.push(s);

            cout << "BFS Traversal: ";
            while (!q.empty()) {
                s = q.front();
                cout << s << " ";
                q.pop();

                for (auto i : adj[s]) {
                    if (!visited[i]) {
                        visited[i] = true;
```

```
                        visited[i] = true;
                        q.push(i);
                    }
                }
            }
            cout << endl;
        }
        // DFS Helper Function (Recursive)
        void DFSUtil(int v, vector<bool> &visited) {
```

```
            visited[v] = true;
            cout << v << " ";
            for (auto i : adj[v]) {
                if (!visited[i])
                    DFSUtil(i, visited);
            }
        }
        // DFS Traversal
        void DFS(int s) {
            vector<bool> visited(V, false);
            cout << "DFS Traversal: ";
            DFSUtil(s, visited);
            cout << endl;
        }
    };
    int main() {
        Graph g(5);
        g.addEdge(0, 1);
        g.addEdge(0, 2);
        g.addEdge(1, 3);
        g.addEdge(1, 4);
        g.addEdge(2, 4);

        g.BFS(0);
        g.DFS(0);

        return 0;
    }
```

Output

```
"c:\Users\Mo7am\Downloads\Data-Structure-Algor
BFS Traversal: 0 1 2 3 4
DFS Traversal: 0 1 3 4 2
```

Time Complexity ($O(V + E)$)

- In both algorithms, every vertex (V) is visited exactly once, and every edge (E) is explored. For a directed graph, it is O(V+E), and for an undirected graph, it is O(V + 2E), which simplifies to the same linear complexity.

Space Complexity ($O(V)$)

- **BFS:** Requires a queue and a visited array, both of which can store up to V vertices.
- **DFS:** Requires a visited array and the recursion stack. In the worst case (a skewed graph like a linked list), the stack depth can reach V.

Conclusion

The choice between BFS and DFS depends on the structure of the data and the goal of your search.

Breadth-First Search (BFS)

- **Strategy:** Explores neighbor nodes first, before moving to the next level neighbors (Level-order).
- **Best For:** Finding the **shortest path** in an unweighted graph. It guarantees finding the closest target.
- **Memory:** Can be memory-intensive if the graph is very wide, as it stores all nodes of the current level in the queue.

Depth-First Search (DFS)

- **Strategy:** Goes as deep as possible down one branch before backtracking.
- **Best For:** Pathfinding in puzzles (like mazes), detecting **cycles** in a graph, and **Topological Sorting**.
- **Memory:** More space-efficient for deep, narrow graphs, but can get stuck in infinite paths if the graph is not handled correctly (visited check is crucial).

Marwadi University Marwadi Chandarana Group NAAC A+	**Marwadi University** **Faculty of Engineering and Technology** **Department of Information and Communication Technology**	
Design and Analysis of Algorithm (01CT0512)	**Aim: Implementing The Sudoku Solution and N-Queens's problem**	
Experiment No:13	**Date: 9/4/2026**	**Enrolment No: 92301733059**

the Sudoku Solution and N-Queens's problem

Theory:

- The Sudoku problem involves filling a 9×9 grid such that each row, each column, and each 3×3 subgrid contains all digits from 1 to 9 without repetition.
- It is solved using the Backtracking algorithm, which is a form of recursion that tries all possible solutions systematically.
- The algorithm searches for an empty cell in the grid.
- It then tries placing digits from 1 to 9 in that cell.
- For each number, it checks whether placing it is valid according to Sudoku rules (row, column, and subgrid constraints).
- If the placement is valid, the algorithm recursively proceeds to fill the next empty cell.
- If no valid number can be placed in a cell, the algorithm backtracks by removing the previously placed number and tries a different value.
- This process continues until the entire grid is filled correctly.
- Backtracking ensures that all possible combinations are explored, guaranteeing a valid solution if one exists.
- Sudoku solving demonstrates constraint satisfaction problems and recursive problem-solving techniques.

Code:

```
#include <iostream>
#include <vector>

using namespace std;

// Check if a queen can be placed on board[row][col]
bool isSafe(vector<vector<int>>& board, int row, int col, int N) {
    // Check this row on left side
    for (int i = 0; i < col; i++)
        if (board[row][i]) return false;

    // Check upper diagonal on left side
    for (int i = row, j = col; i >= 0 && j >= 0; i--, j--)
```

```
        for (int i = row, j = col; i >= 0 && j >= 0; i--, j--)
            if (board[i][j]) return false;

        // Check lower diagonal on left side
        for (int i = row, j = col; j >= 0 && i < N; i++, j--)
            if (board[i][j]) return false;
        return true;
    }
    bool solveNQUtil(vector<vector<int>>& board, int col, int N) {
        if (col >= N) return true; // All queens are placed
        for (int i = 0; i < N; i++) {
            if (isSafe(board, i, col, N)) {
                board[i][col] = 1; // Place queen
                if (solveNQUtil(board, col + 1, N)) return true;
                board[i][col] = 0; // Backtrack
            }
        }
        return false;
    }
    int main() {
        int N = 4;
        vector<vector<int>> board(N, vector<int>(N, 0));
        if (solveNQUtil(board, 0, N)) {
            for (auto row : board) {
                for (int cell : row) cout << (cell ? "Q " : ". ");
                cout << endl;
            }
        }
        return 0;
    }
```

Output

```
"c:\Users\Mo7am\Downloads\Data-Structure-Algorithms-main\D
. . Q .
Q . . .
. . . Q
. Q . .
```

2. N-Queens's problem

```
#include <iostream>
#include <vector>

using namespace std;

bool isSafe(vector<vector<int>>& grid, int row, int col, int num) {
    for (int x = 0; x < 9; x++)
        if (grid[row][x] == num || grid[x][col] == num) return false;

    int startRow = row - row % 3, startCol = col - col % 3;
    for (int i = 0; i < 3; i++)
        for (int j = 0; j < 3; j++)
            if (grid[i + startRow][j + startCol] == num) return false;

    return true;
}

bool solveSudoku(vector<vector<int>>& grid) {
    int row = -1, col = -1;
    bool empty = false;
    for (int i = 0; i < 9; i++) {
        for (int j = 0; j < 9; j++) {
            if (grid[i][j] == 0) {
                row = i; col = j;
                empty = true; break;
            }
        }
        if (empty) break;
    }
```

```
            if (empty) break;
        }

        if (!empty) return true; // Success

        for (int num = 1; num <= 9; num++) {
            if (isSafe(grid, row, col, num)) {
                grid[row][col] = num;
                if (solveSudoku(grid)) return true;
                grid[row][col] = 0; // Backtrack
            }
        }
        return false;
    }
    int main() {
        vector<vector<int>> grid = {
            {3, 0, 6, 5, 0, 8, 4, 0, 0}, {5, 2, 0, 0, 0, 0, 0, 0, 0},
            {0, 8, 7, 0, 0, 0, 0, 3, 1}, {0, 0, 3, 0, 1, 0, 0, 8, 0},
            {9, 0, 0, 8, 6, 3, 0, 0, 5}, {0, 5, 0, 0, 9, 0, 6, 0, 0},
            {1, 3, 0, 0, 0, 0, 2, 5, 0}, {0, 0, 0, 0, 0, 0, 0, 7, 4},
            {0, 0, 5, 2, 0, 6, 3, 0, 0}
        };
        if (solveSudoku(grid)) {
            for (auto r : grid) {
                for (int d : r) cout << d << " ";
                cout << endl;
            }
        }
        return 0;
    }
```

Output

```
"c:\Users\Mo7am\Downloads\Data-Structure
3 1 6 5 7 8 4 9 2
5 2 9 1 3 4 7 6 8
4 8 7 6 2 9 5 3 1
2 6 3 4 1 5 9 8 7
9 7 4 8 6 3 1 2 5
8 5 1 7 9 2 6 4 3
1 3 8 9 4 7 2 5 6
6 9 2 3 5 1 8 7 4
7 4 5 2 8 6 3 1 9
```

Time Complexity ($O(V + E)$)

- **N-Queens:** O(N!) — In the worst case, the algorithm explores all permutations of row placements for N queens.

- **Sudoku:** O(9^{n . n}) — Where n is the number of empty cells. While technically exponential, the constraints of Sudoku (row/col/grid) prune the search tree significantly in practice.

Space Complexity

- **Auxiliary Space:** O(N^2) for N-Queens and O(1) (fixed 9 times 9 grid) for Sudoku. However, the **recursion stack** depth is O(N) for N-Queens and O(81) for Sudoku.

Conclusion

Both problems showcase the efficiency of **Backtracking** over pure Brute Force.

- **Pruning:** The isSafe function acts as a "pruning" mechanism. By checking constraints early, we avoid exploring massive branches of the search tree that are guaranteed to fail.
- **Recursion:** These problems are naturally recursive. We solve one "piece" (a queen's position or a cell's number) and then ask the algorithm to solve the "rest" of the board.
- **State Management:** The "Set-Recurse-Unset" pattern (Backtracking) is vital. If a choice doesn't lead to a solution, we must reset the board state (grid[row][col] = 0) before trying the next option.

Marwadi University Marwadi Chandarana Group NAAC A+	**Marwadi University** **Faculty of Engineering and Technology** **Department of Information and Communication Technology**	
Design and Analysis of Algorithm (01CT0512)	**Aim: Implementing The Backtracking**	
Experiment No:14	**Date: 9/4/2026**	**Enrolment No: 92301733059**

the Backtracking

Theory:

- Backtracking is an algorithmic technique used to solve problems by trying all possible solutions and eliminating those that do not satisfy the given constraints.
- It is a refined form of brute-force approach, where the algorithm builds a solution step by step.
- At each step, it checks whether the current partial solution is valid.
- If the solution is valid, it proceeds further; otherwise, it backtracks by undoing the last step and tries a different option.
- This process continues until a valid solution is found or all possibilities are exhausted.
- Backtracking is typically implemented using recursion.
- It is widely used in problems like N-Queens, Sudoku, Permutations, and Subsets.
- The efficiency of backtracking is improved by pruning, which avoids exploring invalid solutions early.
- It is especially useful for solving Constraint Satisfaction Problems (CSPs).

Code:

```
#include <iostream>
#include <vector>

using namespace std;

// Check if it's safe to place a queen at board[row][col]
bool isSafe(vector<vector<int>>& board, int row, int col, int n) {
    // Check this column on upper side
    for (int i = 0; i < row; i++)
        if (board[i][col]) return false;

    // Check upper left diagonal
```

```
        for (int i = row, j = col; i >= 0 && j >= 0; i--, j--)
            if (board[i][j]) return false;

        // Check upper right diagonal
        for (int i = row, j = col; i >= 0 && j < n; i--, j++)
            if (board[i][j]) return false;

        return true;
    }

    bool solveNQueens(vector<vector<int>>& board, int row, int n) {
        // Base Case: All queens are placed
        if (row >= n) return true;

        // Try placing queen in all columns of the current row
        for (int col = 0; col < n; col++) {
            if (isSafe(board, row, col, n)) {
                board[row][col] = 1; // Place queen

                // Recur to place rest of the queens
                if (solveNQueens(board, row + 1, n))
                    return true;

                // BACKTRACK: Remove queen from board[row][col]
                board[row][col] = 0;
            }
        }
        return false; // If queen cannot be placed in any col of this row
    }

```

```
        for (int i = 0; i < n; i++) {
            for (int j = 0; j < n; j++)
                cout << (board[i][j] ? "Q " : ". ");
            cout << endl;
        }
    }
    int main() {
        int n = 4; // Change this for N-Queens
        vector<vector<int>> board(n, vector<int>(n, 0));
        if (solveNQueens(board, 0, n))
            printBoard(board, n);
        else
            cout << "Solution does not exist";
        return 0;
    }
```

Output

```
"c:\Users\Mo7am\Downloads\Data-Structure-Al
. Q . .
. . . Q
Q . . .
. . Q .
```

Time Complexity

- **Worst Case: O(N!)**
  For the first queen, there are N options. For the second, N-1 (roughly), and so on. While backtracking prunes many branches, the upper bound for N-Queens and similar permutation problems remains factorial or exponential.

- **Average Case:** Significant improvement over pure brute force because the algorithm stops exploring a path (pruning) as soon as it violates a constraint.

Space Complexity

- **Auxiliary Space:** O(N) This is primarily due to the recursion stack. Each recursive call adds a frame to the stack. Additionally, a 1D or 2D array is used to store the positions, which scales with the input size N.ecursion stack depth is O(N) for N-Queens and O(81) for Sudoku.

Conclusion

Backtracking is the preferred method for solving **constraint satisfaction problems** where you need to find one (or all) solutions among a massive set of possibilities.

- **Pruning:** This is the most important feature. By "killing" a branch early (e.g., if two queens are in the same row), you save thousands of unnecessary computations compared to simple brute force.
- **State Management:** The "backtrack" step is critical—you must undo your last move to restore the state of the problem before trying the next option.
- **Common Use Cases:**
  - **Sudoku Solvers:** Filling empty cells while following rules.
  - **Graph Coloring:** Assigning colors to vertices such that no two adjacent vertices share a color.
  - **Hamiltonian Path:** Finding a path that visits every vertex exactly once.